



ΔΙΑΧΕΙΡΙΣΗ ΣΥΝΘΕΤΩΝ ΔΕΔΟΜΕΝΩΝ

Εργασία 1

ΔΗΜΗΤΡΑ ΧΡΙΣΤΙΝΑ ΓΚΑΡΑΒΕΛΑ
ΑΜ: 5051

ΜΕΡΟΣ 1

Λίγα Λόγια για τον Αλγόριθμο

Για το πρώτο μέρος της εργασίας, υλοποιήθηκε ο αλγόριθμος Merge-Join. Ο αλγόριθμος υλοποιεί μόνο το στάδιο της συγχώνευσης αφού τα αρχεία εισόδου (R_sorted.tsv και S_sorted.tsv) είναι ήδη ταξινομημένα. Οι γραμμές διαβάζονται σειριακά από τα δύο αρχεία και συγκρίνονται βάσει του πρώτου πεδίου (join key). Για τη διαχείριση διπλοτύπων και την αποδοτική επεξεργασία, χρησιμοποιείται buffer που αποθηκεύει προσωρινά τις γραμμές του αρχείου S που αντιστοιχούν στο τρέχον join key του αρχείου R. Έτσι, αποφεύγεται η επαναλαμβανόμενη ανάγνωση γραμμών από το αρχείο S όταν υπάρχουν πολλαπλές γραμμές στο R με το ίδιο κλειδί.

Άνοιγμα Αρχείων και Ανάγνωση Δεδομένων

Τα αρχεία ανοίγουν με χρήση `BufferedReader`, και οι γραμμές διαχωρίζονται σε πεδία με χρήση του `tab`. Η εγγραφή των αποτελεσμάτων γίνεται με `BufferedWriter`, με κάθε γραμμή της εξόδου να περιέχει το κοινό πεδίο και τα αντίστοιχα δεύτερα πεδία των R και S.

Η μεταβλητή `currentJoinKey` ελέγχει αν το key έχει αλλάξει και, αν ναι, δημιουργείται νέος buffer. Οι τιμές προστίθενται στο αρχείο εξόδου όταν εντοπίζεται ταύτιση στα πεδία join. Καταγράφεται επίσης το μέγιστο μέγεθος του buffer.

Υλοποίηση του Αλγόριθμου

Αρχικά, ελέγχεται αν το `joinKey` είναι ίσο με το `currentJoinKey`.

- Αν είναι ίσα, δεν διαβάζονται ξανά οι γραμμές από το αρχείο S, αλλά επαναχρησιμοποιούνται οι γραμμές που βρίσκονται ήδη στο buffer.
- Αν είναι νέο Join Key, ενημερώνεται η μεταβλητή `currentJoinKey` με τη νέα τιμή του `joinKey`. Έπειτα, καθαρίζεται το buffer ώστε να αποθηκεύσουμε τις νέες ταιριαστές γραμμές του S που αντιστοιχούν σε αυτό το νέο join key.

```
//αν τα κλειδια είναι ιδια epanaxrhsimopoihse to buffer
// kai grapse thn grammh sto output file
if (currentJoinKey != null && joinKey.equals(currentJoinKey)) {
    for (String rightRow : buffer) {
        String[] fieldsRight = rightRow.split("\t");
        writer.write(fieldsLeft[0] + "\t" + fieldsLeft[1] + "\t" + fieldsRight[1]);
        writer.newLine();
    }
} else { //pare to epomeno kleidi ws currentJoinKey kai adeiase to buffer
    currentJoinKey = joinKey;
    buffer.clear();
}
```

Συγκρίνεται το joinKey με το rightKey:

- `cmp > 0`: Το joinKey είναι "μεγαλύτερο" (lexicographically) από το rightKey, τότε η τρέχουσα γραμμή του S δεν μπορεί να ταιριάζει με το key. Διαβάζουμε την επόμενη γραμμή (`lineRight = readerRight.readLine()`).
- `cmp < 0`: Το joinKey είναι "μικρότερο" από το rightKey, τότε έχουμε περάσει από τις πιθανές ταιριαστές γραμμές και βγαίνουμε από το loop.
- `cmp == 0`: Βρέθηκε ταιριαστή γραμμή στο S. Προσθέτουμε τη γραμμή στο buffer και εισάγουμε τα στοιχεία στο αρχείο εξόδου όπως ανέφερα παραπάνω.

```
// psakse tis grammes tou R gia match me to current key
while (lineRight != null) {
    String[] fieldsRight = lineRight.split("\t");
    String rightKey = fieldsRight[0];
    int cmp = joinKey.compareTo(rightKey);

    //an to key R einai megalitero apo to kleidi ths grammhs tou S tote proxwra sto S
    if (cmp > 0) {
        lineRight = readerRight.readLine();
        //an einai mikrotero tote den ehoume match (tha to eihame vrei afou einai taksinomimena)
    } else if (cmp < 0) {
        break;
    } else { // einai match -> apothikeuse to buffer kai grapse sto output file
        buffer.add(lineRight);
        writer.write(fieldsLeft[0] + "\t" + fieldsLeft[1] + "\t" + fieldsRight[1]);
        writer.newLine();
        lineRight = readerRight.readLine();
    }
}
```

Μετά το πέρας της αναζήτησης για το νέο join key, ελέγχεται αν το μέγεθος του buffer είναι μεγαλύτερο από το `maxSizeOfBuffer` και, αν ναι, ενημερώνεται η μεταβλητή.

```
// enimerwse to megisto megethos gia to buffer if needed
if (buffer.size() > maxSizeOfBuffer) {
    maxSizeOfBuffer = buffer.size();
}
```

Εκτέλεση του Προγράμματος

```
javac MergeJoin.java
```

```
java MergeJoin R_sorted.tsv S_sorted.tsv RjoinS.tsv
```

ΜΕΡΟΣ 2

Λίγα Λόγια για τον Αλγόριθμο

Στο πρόγραμμα Union υλοποιείται η ένωση (union) των δύο ταξινομημένων αρχείων R_sorted.tsv και S_sorted.tsv, θεωρώντας ότι διαθέτουν ακριβώς τα ίδια πεδία. Η ένωση R U S περιλαμβάνει όλες τις γραμμές που υπάρχουν είτε στο R είτε στο S. Το πρόγραμμα αποφεύγει τα διπλότυπα τόσο κατά την ανάγνωση όσο και κατά την εγγραφή.

Άνοιγμα Αρχείων και Ανάγνωση Δεδομένων

Το άνοιγμα και η ανάγνωση των αρχείων είναι παρόμοια με το πρώτο μέρος. Δηλαδή διαβάζει μία γραμμή τη φορά από κάθε αρχείο (με χρήση `BufferedReader` και `readLine()`).

Υλοποίηση του Αλγορίθμου

Εάν και τα δύο αρχεία έχουν διαθέσιμες γραμμές, συγκρίνονται οι τρέχουσες γραμμές για να ξέρουμε ποια θα πάρουμε ως επόμενη (χρησιμοποιώντας την `compareTo()`):

- Αν `sLine == null || rLine.compareTo(sLine) <= 0`: το `rLine` είναι μικρότερο ή ίσο από το `sLine` ή το S έχει τελειώσει (είναι null), τότε πάρε το `rLine` ως υποψήφια γραμμή για εγγραφή (candidate) και προχώρησε στην επόμενη γραμμή του R
- Αν `rLine.compareTo(sLine) > 0` τότε είτε το R τελείωσε, είτε το `rLine` είναι μεγαλύτερο, πάρε το `sLine` ως candidate και διάβασε την επόμενη γραμμή του S.

Αυτό συμβαίνει έτσι ώστε κάθε φορά να παίρνουμε την μικρότερη γραμμή και να μην χάνεται κάποια εγγραφή.

- `lastWritten == null || !candidate.equals(lastWritten)` : Αν δεν έχουμε γράψει καμία γραμμή ακόμα ή αν η candidate διαφέρει από την τελευταία γραμμή που γράψαμε (`lastWritten`) τότε γράψε τη γραμμή στο αρχείο εξόδου και ενημέρωσε τη `lastWritten`. Έτσι, αποφεύγονται τα διπλότυπα.

```

//an rline <= sline -> pare to rline gia candidate kai proxwra sto R arxeio
if (rline != null && (sline == null || rline.compareTo(sline) <= 0)) {
    candidate = rline;
    rline = readerR.readLine();
} else { // an einai megalitero pare to sline gia candidate kai proxwra sto S arxeio
    candidate = sline;
    sline = readerS.readLine();
}

// elegxos gia duplicate
if (lastWritten == null || !candidate.equals(lastWritten)) {
    writer.write(candidate);
    writer.newLine();
    lastWritten = candidate;
}

```

Εκτέλεση του Προγράμματος

```
javac Union.java
```

```
java Union R_sorted.tsv S_sorted.tsv RunionS.tsv
```

ΜΕΡΟΣ 3

Λίγα Λόγια για τον Αλγόριθμο

Ο αλγόριθμος Intersection υπολογίζει την τομή δύο ταξινομημένων αρχείων κειμένου (R_sorted.tsv και S_sorted.tsv), δηλαδή τις κοινές γραμμές που υπάρχουν και στα δύο.

Άνοιγμα Αρχείων και Ανάγνωση Δεδομένων

Το άνοιγμα και η ανάγνωση των αρχείων είναι παρόμοια με το πρώτο μέρος. Δηλαδή, διαβάζει μία γραμμή τη φορά από κάθε αρχείο (με χρήση BufferedReader και readLine. Οι διπλότυπες γραμμές παραλείπονται με χρήση της μεθόδου skipDuplicates.

Υλοποίηση του Αλγορίθμου

Εάν και τα δύο αρχεία έχουν διαθέσιμες γραμμές, συγκρίνονται οι τρέχουσες γραμμές (χρησιμοποιώντας την compareTo()):

- Αν $cmp < 0$ ($rLine < sLine$), σημαίνει ότι η γραμμή υπάρχει μόνο στο R οπότε παραλείπεται. Παραλείπουμε όλες τις ίδιες γραμμές της R (αν έχει διπλότυπα) με τη `skipDuplicates`.
- Αν $cmp > 0$ ($rLine > sLine$), σημαίνει ότι η γραμμή υπάρχει μόνο στο S οπότε παραλείπεται. Παραλείπουμε όλες τις ίδιες γραμμές της S με τη `skipDuplicates`.
- Αν είναι ίδιες, η γραμμή υπάρχει και στα δύο αρχεία . Τότε ελέγχεται αν δεν έχει ήδη γραφτεί στο output (`lastWritten`) και αν όχι, γράφεται. Προχωράμε και από τα δύο αρχεία στην επόμενη διαφορετική γραμμή, ώστε να παρακάμψουμε τα διπλότυπα.

```
if (cmp < 0) {
    rLine = skipDuplicates(readerR, rLine);
} else if (cmp > 0) { //megaliterh
    sLine = skipDuplicates(readerS, sLine);
} else { // idies ara ehoume koino stoixeio -> grafetai sto output
    if (lastWritten == null || !rLine.equals(lastWritten)) {
        writer.write(rLine);
        writer.newLine();
        lastWritten = rLine;
    }
    rLine = skipDuplicates(readerR, rLine);
    sLine = skipDuplicates(readerS, sLine);
}
```

Με τη βοήθεια της μεθόδου `skipDuplicates()`, παραλείπονται επαναλαμβανόμενες γραμμές στην είσοδο. Συγκεκριμένα, ελέγχεται αν η νέα γραμμή είναι ίδια με την προηγούμενη (`line.equals(current)`), αν όχι τότε επιστρέφεται και γίνεται η επόμενη `current`. Επιστρέφει `null` όταν τελειώσει το αρχείο.

```
private String skipDuplicates(BufferedReader reader, String current) throws IOException {
    String line;
    while ((line = reader.readLine()) != null) {
        if (!line.equals(current)) {
            return line;
        }
    }
    return null;
}
```

Εκτέλεση του Προγράμματος

`javac Intersection.java`

`java Intersection R_sorted.tsv S_sorted.tsv RintersectionS.tsv`

ΜΕΡΟΣ 4

Λίγα Λόγια για τον Αλγόριθμο

Ο αλγόριθμος SetDifference υπολογίζει τη διαφορά R - S μεταξύ δύο ταξινομημένων αρχείων R_sorted.tsv και S_sorted.tsv. Η διαφορά R - S περιλαμβάνει όλες τις γραμμές που υπάρχουν στο R αλλά όχι στο S.

Άνοιγμα Αρχείων και Ανάγνωση Δεδομένων

Το άνοιγμα και η ανάγνωση των αρχείων είναι παρόμοια με το πρώτο μέρος. Δηλαδή, διαβάζει μία γραμμή τη φορά από κάθε αρχείο (με χρήση BufferedReader και readLine. Οι διπλότυπες γραμμές παραλείπονται με χρήση της μεθόδου skipDuplicates.

Υλοποίηση του Αλγορίθμου

Εάν και τα δύο αρχεία έχουν διαθέσιμες γραμμές, συγκρίνονται οι τρέχουσες γραμμές (χρησιμοποιώντας την compareTo()):

- Αν `cmp < 0`, σημαίνει ότι η γραμμή υπάρχει μόνο στο R, οπότε γράφεται στην έξοδο (αν δεν είναι διπλότυπη). Έπειτα, παραλείπονται τα πιθανά διπλότυπα από το R.
- Αν `cmp == 0`, σημαίνει ότι η γραμμή υπάρχει και στα δύο αρχεία οπότε παραλείπεται και από τα δύο, γιατί δεν ανήκει στη διαφορά. Εδώ ο αλγόριθμος καλεί `skipDuplicates()` και στα δύο αρχεία.
- Αλλιώς αν `cmp > 0`, σημαίνει ότι η τιμή στην γραμμή του S είναι μικρότερη, άρα προχωράμε στο επόμενο `sLine` για να κάνουμε την σύγκριση ξανά.

```
if (cmp < 0) {
    // rLine mono sto R
    if (lastWritten == null || !rLine.equals(lastWritten)) {
        writer.write(rLine);
        writer.newLine();
        lastWritten = rLine;
    }
    rLine = skipDuplicates(readerR, rLine);
} else if (cmp == 0) {
    // koih grammh kai sta dyo -> skip both
    rLine = skipDuplicates(readerR, rLine);
    sLine = skipDuplicates(readerS, sLine);
} else {
    // sLine < rLine proxwra parakatw sto S
    sLine = skipDuplicates(readerS, sLine);
}
```

Αν έχουν απομείνει γραμμές στο R, σημαίνει ότι δεν υπάρχουν στο S, οπότε γράφονται στην έξοδο (αν δεν είναι διπλότυπες).

```
// οι υπολοιπες γραμμες του R einai pithanη apotelesma
while (rLine != null) {
    if (lastWritten == null || !rLine.equals(lastWritten)) {
        writer.write(rLine);
        writer.newLine();
        lastWritten = rLine;
    }
    rLine = skipDuplicates(readerR, rLine);
}
```

Με τη βοήθεια της μεθόδου `skipDuplicates()`, παραλείπονται επαναλαμβανόμενες γραμμές στην είσοδο όπως και στο 3^ο μέρος.

Εκτέλεση του Προγράμματος

```
javac SetDifference.java
```

```
java SetDifference R_sorted.tsv S_sorted.tsv RdifferenceS.tsv
```

ΜΕΡΟΣ 5

Λίγα Λόγια για τον Αλγόριθμο

Ο αλγόριθμος `RGroupBySum` βασίζεται στον αλγόριθμο `Sort-Merge`, ο οποίος λειτουργεί διαιρώντας επαναληπτικά έναν πίνακα σε μικρότερα υποσύνολα, εκτελώντας αναδρομικά ταξινόμηση στα υποσύνολα και στη συνέχεια συγχωνεύοντάς τα σε ένα ενιαίο ταξινομημένο πίνακα. Η διαφορά στον αλγόριθμο μας είναι στον τρόπο συγχώνευσης. Συγκεκριμένα, όταν δύο πλειάδες (`key`, `value`) έχουν το ίδιο `key`, δεν προστίθενται ξεχωριστά. Αντίθετα, συγχωνεύονται σε μία πλειάδα, με το ίδιο `key` και ως `value` το άθροισμα των αρχικών τιμών.

Άνοιγμα Αρχείων και Ανάγνωση Δεδομένων

Η ανάγνωση γίνεται γραμμή-γραμμή με χρήση της κλάσης `BufferedReader`, και τα δεδομένα αποθηκεύονται σε δύο ξεχωριστές λίστες `List<String> keys` και `List<Integer> values`, οι οποίες αποθηκεύουν τα κλειδιά και τις αριθμητικές τιμές, αντίστοιχα. Η αντιστοιχία μεταξύ των στοιχείων διατηρείται μέσω του `index`: π.χ. το `keys.get(i)` αντιστοιχεί στο `values.get(i)`.

Υλοποίηση του Αλγορίθμου

Αρχικά, υλοποίησα την μέθοδο `mergeSortGroup`, η οποία υλοποιεί αναδρομικά την λογική του Sort-Merge και την κάλεσα αρχικά με παραμέτρους **keys** (η λίστα με όλα τα key που διαβάστηκαν από το αρχείο), **values** (η λίστα με τα αντίστοιχα value για κάθε key), **0** (η αρχή της λίστας), **keys.size() - 1** (το τέλος της λίστας), **groupedKeys** (κενή λίστα για να αποθηκεύσουμε τα τελικά keys μετά το sort+group) και **groupedValues** (κενή λίστα για να αποθηκεύσουμε τα τελικά values μετά το sort+sum).

```
27 mergeSortGroup(keys, values, 0, keys.size() - 1, groupedKeys, groupedValues);
```

Ελέγχουμε αν το τμήμα της λίστας που εξετάζουμε περιέχει μόνο ένα στοιχείο, το οποίο προστίθεται στη λίστα αποτελέσματος (`left==right`).

```
49 if (left == right) {  
50     groupedKeys.add(keys.get(left));  
51     groupedValues.add(values.get(left));  
52     return;  
53 }
```

Αν όχι, τότε η λίστα χωρίζεται σε δύο υπολίστες από `left` έως `mid` και από `mid+1` έως `right`. Η αναδρομή καλεί πρώτα την αριστερή και έπειτα τη δεξιά υπολίστα, ώστε να φτάσει μέχρι το αριστερότερο στοιχείο (φύλλο του δέντρου), πριν επιστρέψει προς τα πάνω για συγχώνευση. Τέλος, καλείται η `mergeGroupedLists()`, η οποία συγχωνεύει τα ταξινομημένα αποτελέσματα.

```
59 List<String> leftKeys = new ArrayList<>();  
60 List<Integer> leftValues = new ArrayList<>();  
61 //System.out.println("EDW KALOUME mergeSortGroupt gia left");  
62 mergeSortGroup(keys, values, left, mid, leftKeys, leftValues);  
63  
64 List<String> rightKeys = new ArrayList<>();  
65 List<Integer> rightValues = new ArrayList<>();  
66 //System.out.println("EDW KALOUME mergeSortGroupt gia right");  
67 mergeSortGroup(keys, values, mid + 1, right, rightKeys, rightValues);  
68  
69 //System.out.println(" MERGE left [" + left + "-" + mid + "] me right [" + (mid+1) + "-" + right + "]);  
70 mergeGroupedLists(leftKeys, leftValues, rightKeys, rightValues, groupedKeys, groupedValues);  
71 //System.out.println(" RESULT OF MERGE " + groupedKeys + " -> " + groupedValues);
```

Η `mergeGroupedLists()` συγκρίνει δύο ταξινομημένες λίστες (`keys1/values1` και `keys2/values2`) και επιστρέφει μία συγχωνευμένη λίστα με ταξινομημένα αποτελέσματα με βάση το key. Συγκεκριμένα, αν

- `key1 < key2`: προστίθεται η εγγραφή από την αριστερή λίστα.
- `key1 > key2`: προστίθεται η εγγραφή από τη δεξιά λίστα.

- $key1 == key2$: γίνεται συγχώνευση και προστίθεται μόνο μία εγγραφή με άθροισμα των τιμών.

Αυτό συμβαίνει επειδή οι `keys1` και `keys2` είναι ήδη ταξινομημένες, οπότε αν το $k1 < k2$, τότε το `k1` πρέπει να έρθει πριν το `k2` στην τελική λίστα, ώστε να διατηρηθεί η σωστή σειρά.

```

91         if (k1.compareTo(k2) < 0) {
92             //System.out.println("Krataw : " + k1 + "\t" + v1);
93             mergedKeys.add(k1);
94             mergedValues.add(v1);
95             i++;
96         } else if (k1.compareTo(k2) > 0) {
97             //System.out.println("Krataw : " + k2 + "\t" + v2);
98             mergedKeys.add(k2);
99             mergedValues.add(v2);
100            j++;
101        } else {
102            int sum = v1 + v2;
103            //System.out.println("Merge: " + k1 + "\t" + v1 + " + " + v2 + " -> " + sum);
104            mergedKeys.add(k1);
105            mergedValues.add(sum);
106            i++;
107            j++;
108        }

```

Μετά το βασικό `while (i < keys1.size() && j < keys2.size())` όπου συγχωνεύουμε μέχρι να τελειώσει τουλάχιστον μία από τις δύο λίστες, μένουν κάποια υπολειπόμενα στοιχεία στην άλλη λίστα (αν δεν έχουν τελειώσει ταυτόχρονα). Αφού ξέρουμε ότι οι λίστες είναι ήδη ταξινομημένες, ό,τι έχει απομείνει, είναι μεγαλύτερο από όλα όσα έχουν ήδη προστεθεί. Συγκεκριμένα, αν η δεξιά λίστα τελείωσε πρώτη, προστίθενται όλα τα υπόλοιπα στοιχεία από την αριστερή.

```

111         //prosthiki ypoloipwn
112         while (i < keys1.size()) {
113             //System.out.println("Prosthetw ypoloipo apo left: " + keys1.get(i) + "\t" + values1.get(i));
114             mergedKeys.add(keys1.get(i));
115             mergedValues.add(values1.get(i));
116             i++;
117         }

```

Αν η αριστερή τελείωσε πρώτη, προστίθενται όλα τα υπόλοιπα στοιχεία από τη δεξιά.

```

119         while (j < keys2.size()) {
120             //System.out.println("Prosthetw ypoloipo apo right: " + keys2.get(j) + "\t" + values2.get(j));
121             mergedKeys.add(keys2.get(j));
122             mergedValues.add(values2.get(j));
123             j++;
124         }

```

Το αποτέλεσμα είναι ταξινομημένη λίστα με μοναδικά `keys` και αθροισμένες `values`.

Εκτέλεση του Προγράμματος

```
javac RGroupBySum.java
```

```
java RGroupBySum.java
```